AWS Lambda Lab 3: Automatic Email Security Alerts with Amazon SNS

1. Lab overview

Project: S3 File Security Alert System

Level: Beginner

Method: AWS Management Console

Estimated duration: 30–45 minutes

Objective

Build a system that checks filenames uploaded to Amazon S3 and sends an email alert when a filename ends with a configured suspicious extension.

The system uses:

- **Amazon S3:** Stores uploaded files and generates events.
- **AWS Lambda:** Checks filenames and decides whether an alert is needed.
- **Amazon SNS:** Sends email notifications.
- **AWS IAM:** Controls which AWS resources the Lambda function can access.
- **Amazon CloudWatch:** Stores execution logs for troubleshooting.

Expected result

When a file named `test.exe` is uploaded to the S3 bucket, Lambda detects its extension and publishes an alert to SNS. SNS sends the alert to the confirmed email subscriber.

When a file named `notes.txt` is uploaded, Lambda records that no alert is required.

Security limitation: This lab checks filenames only. It does not inspect file contents, detect malware, or prove that a file is safe or malicious.

---

2. Architecture

The workflow follows this sequence:

1. A file is uploaded to Amazon S3.
2. S3 sends an object-created event to Lambda.
3. Lambda extracts the filename from the event.
4. Lambda compares the filename extension with a list of configured extensions.
5. If the extension matches, Lambda publishes a message to an SNS topic.

6. SNS emails the message to the confirmed subscriber.
7. Lambda execution details and printed messages are available in CloudWatch Logs.

Architecture flow:

`Amazon S3 → AWS Lambda → Amazon SNS → Email inbox`

CloudWatch Logs records the Lambda execution and helps troubleshoot errors.

---

3. Prerequisites

Before beginning, make sure the following are available:

- An AWS account with permission to manage Lambda, S3, SNS, and IAM.
- An existing S3 bucket named `zelos-df3`.
- An existing Lambda function that is already triggered by uploads to `zelos-df3`.
- Access to an email inbox for receiving and confirming SNS notifications.

Use the same AWS Region as the existing Lambda function and S3 bucket for this lab.

Important design decision

The existing S3-triggered Lambda function will be updated to send SNS alerts. This avoids creating an additional S3 notification with overlapping event filters.

Use the existing file-extension-checking function from Lab 2 if it is available. If the existing function is named `zelos-s3-file-detector`, that function can be updated instead.

Do not delete the existing function or bucket.

---

4. Step 1 — Create an SNS topic

An SNS topic is a communication channel through which AWS services can publish messages to subscribers.

Console procedure

1. Sign in to the AWS Management Console.
2. Search for **Amazon SNS**.
3. Open the SNS console.
4. Select **Topics** from the navigation menu.
5. Choose **Create topic**.

6. Select **Standard** as the topic type.
7. Enter the topic name:
   `zelos-file-security-alerts`
8. Leave optional settings at their defaults for this introductory lab.
9. Choose **Create topic**.

Record the topic ARN

After the topic is created, open its details page and copy its Amazon Resource Name (ARN).

It will look similar to this:

`arn:aws:sns:us-east-1:123456789012:zelos-file-security-alerts`

This is only an example. Use the actual ARN displayed in the AWS console.

Why this is needed: Lambda must know which SNS topic should receive the alert messages.

---

5. Step 2 — Subscribe an email address

An SNS subscription tells the topic where its messages should be delivered.

Console procedure

1. Open the SNS topic `zelos-file-security-alerts`.
2. Choose **Create subscription**.
3. For **Protocol**, select `Email`.
4. For **Endpoint**, enter an email address that can be accessed.
5. Choose **Create subscription**.
6. Open the email inbox.
7. Find the subscription confirmation email from AWS Notifications.
8. Open the message and choose **Confirm subscription**.

Return to the SNS console and verify that the subscription is confirmed rather than pending confirmation.

Why this is needed: SNS does not deliver normal topic notifications to an email subscription until the recipient confirms it.

---

6. Step 3 — Test SNS email delivery

Before changing Lambda, test the email service independently.

Console procedure

1. Open the `zelos-file-security-alerts` topic.
2. Choose **Publish message**.
3. Enter the subject:
   `SNS Test Alert`
4. Enter the message:
   `This is a test notification from the Zelos file security alert lab.`
5. Choose **Publish message**.
6. Check the subscribed email inbox.

Expected result

The email inbox receives the test message.

If the message does not arrive, check the SNS subscription status and the spam or junk folder before continuing.

Why this is needed: Testing SNS separately confirms that email notifications work before Lambda is introduced into the workflow.

---

7. Step 4 — Give Lambda permission to publish to SNS

Lambda uses an execution role to access AWS services. The role must have permission to publish messages to the specific SNS topic.

Console procedure

1. Open **AWS Lambda**.
2. Open the existing S3-triggered file-checking function.
3. Select **Configuration**.
4. Choose **Permissions**.
5. Find the execution role name and open the role in IAM.

The role usually has a name similar to `zelos-s3-file-detector-role-...`. Use the actual role attached to the function.

Add an inline IAM policy

1. In IAM, open the execution role.
2. Choose **Add permissions**.
3. Select **Create inline policy**.

4. Open the **JSON** policy editor.
5. Enter the following policy, replacing the example resource ARN with the actual ARN of the SNS topic.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublishFileSecurityAlerts",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:us-east-1:123456789012:zelos-file-
security-alerts"
    }
  ]
}
```

6. Choose **Next**.
7. Enter the policy name:
   `ZelosPublishFileSecurityAlerts`
8. Choose **Create policy** or the equivalent save option shown in the console.

What the policy means

- `Effect: Allow` permits the specified action.
- `Action: sns:Publish` permits publishing messages to SNS.
- `Resource` limits the permission to the selected SNS topic.

The example ARN must be replaced with the real ARN. Do not use a wildcard resource when a specific topic ARN is available.

Why this is needed: Without the permission, Lambda may detect a suspicious filename but fail when it tries to publish the alert.

---

8. Step 5 — Configure the SNS topic ARN in Lambda

The function needs the SNS topic ARN at runtime. Store it as a Lambda environment variable rather than placing it directly into the code.

Console procedure

1. Return to the existing Lambda function.
2. Select **Configuration**.

3. Open **Environment variables**.
4. Choose **Edit**.
5. Add the following variable:
6. Save the changes.

Why this is needed: Environment variables let the function use the correct topic without embedding resource configuration in the code.

The topic ARN is not a password or secret, but keeping configuration separate from code makes the function easier to maintain.

---

9. Step 6 — Update the Lambda code

Open the function's **Code** tab and select `lambda_function.py`.

Save a backup of the current code before replacing it. Then use the following code for this lab.

```
import os
import boto3
from urllib.parse import unquote_plus

sns = boto3.client("sns")

SUSPICIOUS_EXTENSIONS = (
    ".exe",
    ".js",
    ".bat",
    ".ps1",
    ".sh"
)


def lambda_handler(event, context):
    topic_arn = os.environ.get("SNS_TOPIC_ARN")

    if not topic_arn:
        raise ValueError("SNS_TOPIC_ARN environment variable is
missing")

    records = event.get("Records", [])
    alerts_sent = 0
```

```
    if not records:
        print("No S3 records found in the event.")
        return {
            "status": "No S3 records",
            "alerts_sent": 0
        }

    for record in records:
        if record.get("eventSource") != "aws:s3":
            continue

        bucket = record["s3"]["bucket"]["name"]
        key = unquote_plus(record["s3"]["object"]["key"])

        filename = key.rsplit("/", 1)[-1]
        filename_lower = filename.lower()

        print(f"Checking file: {filename}")

        if filename_lower.endswith(SUSPICIOUS_EXTENSIONS):
            message = (
                "FILE SECURITY ALERT\n\n"
                f"Bucket: {bucket}\n"
                f"Object key: {key}\n"
                f"Filename: {filename}\n"
                "Reason: The filename matches a configured "
                "extension rule.\n"
                "Action: Review the file according to your "
                "security procedure.\n\n"
                "This is a filename-based alert, not a malware verdict."
            )

            sns.publish(
                TopicArn=topic_arn,
                Subject="S3 File Security Alert",
                Message=message
            )

            alerts_sent += 1
            print(f"Alert published for: {filename}")
```

```
            else:
                print(f"No extension alert required for: {filename}")

        return {
            "status": "Completed",
            "alerts_sent": alerts_sent
        }
```

Understand the important parts

SNS client

`boto3.client("sns")` creates the AWS SDK client used to publish messages.

Environment variable

`os.environ.get("SNS_TOPIC_ARN")` reads the topic ARN configured in the Lambda console.

S3 event records

The `Records` list contains details of objects that generated the event.

Filename decoding

`unquote_plus()` decodes URL-encoded object keys from S3 events, including spaces represented by plus signs.

Extension comparison

`endswith(SUSPICIOUS_EXTENSIONS)` checks whether the filename ends with one of the configured extensions. The comparison is case-insensitive because the filename is converted to lowercase.

Publishing the alert

`sns.publish()` sends the message to the configured SNS topic. SNS handles delivery to the confirmed email subscriber.

Return value

The returned dictionary reports that processing completed and how many alerts were published.

Deploy the code

1. Choose **Deploy** in the Lambda code editor.
2. Wait for the success message.

3. Confirm that the environment variable and execution role policy are saved.

---

10. Step 7 — Confirm the S3 trigger

The existing Lambda function must still have its S3 trigger configured.

Console procedure

1. Open the Lambda function.
2. Check the function's overview or **Configuration** → **Triggers**.
3. Confirm that Amazon S3 is listed as a trigger.
4. Verify that the source bucket is `zelos-df3`.
5. Confirm that the trigger listens for the appropriate object-created events.

If the existing trigger is already working, do not add another one.

If a trigger must be configured, use the Lambda console's S3 trigger setup and follow its permission prompts. Avoid creating overlapping S3 notification filters for the same event types.

Why this is needed: SNS does not monitor S3 uploads itself in this design. The S3 event invokes Lambda, and Lambda decides whether to publish a notification.

---

11. Step 8 — Test with a suspicious filename

Use a harmless test file. Do not upload real malware.

Test A: Suspicious extension

1. Create a harmless text file on the computer.
2. Name it `test.exe`.
3. The file can contain a simple line such as:
   `This is a harmless filename detection test.`
4. Open the S3 console.
5. Open the bucket `zelos-df3`.
6. Choose **Upload**.
7. Upload `test.exe`.
8. Wait a short time for the S3 event and Lambda execution.
9. Check the subscribed email inbox.

Expected result

The inbox receives an email with the subject:

`S3 File Security Alert`

The message includes the bucket, object key, filename, and reason for the alert.

The Lambda execution should also report that an alert was published.

Important: The filename is only a test. The system does not execute the file.

---

12. Step 9 — Test with a normal filename

Upload a harmless file named:

`notes.txt`

Expected result

- Lambda runs after the upload.
- The function checks the filename.
- The `.txt` extension does not match the configured list.
- CloudWatch logs record that no extension alert is required.
- No security-alert email is published for this file.

The function still runs for the upload; it simply does not publish an alert.

---

13. Step 10 — Check CloudWatch Logs

CloudWatch Logs provides the record of the Lambda execution.

Console procedure

1. Open the Lambda function.
2. Select **Monitor**.
3. Choose **View CloudWatch logs**.
4. Open the newest log stream.
5. Look for messages such as:

For a flagged filename:

```
Checking file: test.exe
Alert published for: test.exe
```

For a normal filename:

```
Checking file: notes.txt
No extension alert required for: notes.txt
```

The precise log stream name and additional messages may differ.

What to look for if an execution fails

- `AccessDenied` or `AuthorizationError`: Check the Lambda execution role and the SNS topic ARN in the policy.
- `SNS_TOPIC_ARN environment variable is missing`: Check the Lambda environment variable.
- No Lambda execution appears: Check the S3 trigger, bucket, Region, and event type.
- Lambda succeeds but email is missing: Check SNS subscription confirmation and the email's spam folder.
- `ResourceNotFound` or a similar resource error: Verify the topic ARN and Region.

---

14. Final verification checklist

Mark each item after confirming it in the AWS console.

- SNS topic `zelos-file-security-alerts` exists.
- Email subscription is confirmed.
- SNS test message arrives in the inbox.
- Lambda execution role allows `sns:Publish` to the intended topic only.
- Lambda environment variable `SNS_TOPIC_ARN` contains the correct ARN.
- Lambda code has been deployed.
- The existing S3 trigger remains enabled for `zelos-df3`.
- Uploading `test.exe` generates an email alert.
- Uploading `notes.txt` does not generate an alert.
- CloudWatch logs show the function's decisions.
- No real malware was uploaded or executed.

---

15. Cleanup

After completing the lab:

1. Remove the two harmless test files from `zelos-df3` if they are no longer needed.
2. If the lab is no longer needed, remove the SNS email subscription.
3. Delete the SNS topic if it was created only for this lab.
4. Remove the `sns:Publish` inline policy if the Lambda function no longer needs to send alerts.

5. Remove the `SNS_TOPIC_ARN` environment variable if the alert feature is removed from the function.
6. If the function is restored to its previous code, verify that its configuration and S3 trigger still work as intended.

Do not delete a shared S3 bucket, a production function, or an IAM role that is used by other workloads.

Review AWS pricing before leaving resources running. Charges can depend on service usage and account eligibility.

---

16. What this lab demonstrates

This project demonstrates how multiple AWS services work together:

- S3 generates an event when a file is uploaded.
- Lambda processes the event without requiring a server to be managed.
- IAM grants the function only the permission it needs to publish alerts.
- SNS distributes a message to an email subscriber.
- CloudWatch Logs provides visibility into the function's execution.

The result is a basic event-driven notification system that can be extended with more advanced security controls.

Further improvements

Possible next steps include:

- Sending alerts to multiple subscribers.
- Adding a configurable extension list.
- Recording alert metadata in DynamoDB.
- Adding an actual file-scanning service for malware detection.
- Using Amazon GuardDuty or other suitable security services for supported threat-detection use cases.

A filename extension rule should remain a simple warning mechanism, not be treated as a malware-detection engine.

Official AWS references

- Process Amazon S3 event notifications with Lambda
- Create an Amazon SNS topic
- Amazon SNS email subscriptions

- AWS Lambda execution roles and SNS permissions
- S3 notification object-key filtering rules